

Campus Resource & Event Portal

Submitted in partial fulfillment of the requirements of the degree of

BACHELOR OF COMPUTER ENGINEERING

by

Mahika Desai - 24102033

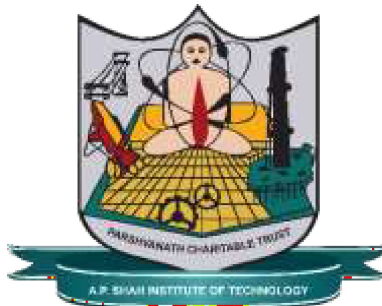
Aarya Bhoir - 24102047

Neha Bagal - 24102050

Archita Dedhia - 24102073

Guide:

Prof. Selvin Furtado



Department of Computer Engineering

A. P. SHAH INSTITUTE OF TECHNOLOGY, THANE

2025-26



Parshvanath Charitable Trust's
A. P. SHAH INSTITUTE OF TECHNOLOGY
(Approved by AICTE New Delhi & Govt. of Maharashtra, Affiliated to University of Mumbai)
(Religious Jain Minority)

CERTIFICATE

This is to certify that the Mini Project entitled “**Campus Resource & Event Portal** is a bonafide work of “**Mahika Desai(24102033), Aarya Bhoir(24102047), Neha Bagal (24102050), Archita Dedhia(24102073),**” submitted to the University of Mumbai in partial fulfillment of the requirement for the award of the degree of **Bachelor of Engineering in Computer Engineering.**

Guide
Prof. Prof. Selvin Furtado

Project Coordinator
Prof. Deepali Kayande

Head of Department
Dr. S.H. Malave



Parshvanath Charitable Trust's
A. P. SHAH INSTITUTE OF TECHNOLOGY
(Approved by AICTE New Delhi & Govt. of Maharashtra, Affiliated to University of Mumbai)
(Religious Jain Minority)

Project Report Approval for Mini Project

This project report, entitled “**Campus Resource & Event Portal**” by *Mahika Desai, Aarya Bhoir, Neha Bagal, Archita Dedhia* is approved for the partial fulfilment of the degree of *Bachelor of Engineering in Computer Engineering, 2025-26*

Examiner Name

Signature

1. _____

2. _____

Date:

Place:

Declaration

We declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, I have adequately cited and referenced the sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Mahika Desai – 24102033

Aarya Bhoir – 24102047

Neha Bagal – 24102050

Archita Dedhia – 24102073

Date:

Abstract

The increasing number of campus activities in educational institutions has created challenges in managing event information, communication, and resource allocation effectively. Traditional methods, which rely on fragmented platforms such as emails and messaging applications, often lead to poor coordination, missed announcements, and scheduling conflicts. To address these issues, this project aims to design and develop a centralized Campus Event Management Portal that streamlines event creation, registration, and communication between students and organizers.

The system is developed using a modern web development stack, with React for the frontend, supported by backend services for efficient data handling, and a secure database for reliable storage. It includes key features such as event categorization, email notifications, calendar integration, and real-time updates, ensuring a seamless user experience.

The results of implementing this system show significant improvements in event management efficiency, increased user engagement, and a reduction in duplication and miscommunication. Additionally, the portal enhances transparency by providing detailed reports on registrations and participation metrics. In conclusion, the proposed system offers a scalable, user-friendly solution that improves the organization and accessibility of campus events. It also aligns with Sustainable Development Goals by promoting inclusive access to information and efficient institutional management, specifically supporting SDG 4 (Quality Education), SDG 9 (Industry, Innovation and Infrastructure) , and SDG 16 (Peace, Justice, and Strong Institutions).

Keywords: Campus Event Management, Event Portal, Centralized System, Student Engagement, Event Registration, Notification System, Calendar Integration, Web Application, Resource Management, Digital Transformation.

CONTENTS

Sr. No.	Chapter Name	Page No.
1	Introduction	
2	Existing System Study	
3	Problem Statement, Objective & Scope	
4	Proposed System	
5	Project Plan	
6	Experimental Setup Technology Stack Used Database Design (ER Diagram, Schema, Normalization) Module Description)	
7	Implementation Details (Front end & Database Integration and SQL Queries & Implementation)	
8	Results (Screenshots)	
9	Conclusion	
10	References	

LIST OF FIGURES

Sr. No.	Figure Name	Page No.
1	Gantt Chart	
2	Architecture Diagram	
3	Entity Relationship Diagram	
4	Data Dictionary	
5	Relational Schema	
6	Database Constraints (if any)	
7	Sample SQL queries	
8		
9		
10		

Chapter 1

Introduction

In modern educational institutions, the management of campus events, academic resources, and organizational activities requires efficient coordination across multiple stakeholders, including students, faculty, and administrative staff. Despite the increasing reliance on digital tools, many campuses continue to operate using fragmented communication channels such as emails, messaging platforms, spreadsheets, and manual booking systems. These disjointed mechanisms lack interoperability and real-time synchronization, leading to critical issues such as scheduling conflicts, duplicate resource bookings, inconsistent data records, and missed event notifications.

The absence of a unified and structured system introduces inefficiencies in both operational workflows and user engagement. Event organizers often face challenges in coordinating logistics, verifying resource availability, and ensuring timely communication with participants. Similarly, students and faculty may struggle to access reliable and up-to-date information regarding ongoing or upcoming events. These limitations highlight the need for a centralized, scalable, and robust platform capable of integrating various campus activities into a single cohesive system.

This project addresses the aforementioned challenges by designing and implementing a centralized Campus Event and Resource Management System. The proposed solution aims to provide a comprehensive digital platform that enables seamless event creation, scheduling, registration, and communication, while ensuring data consistency and system reliability. The system is designed with a modular architecture that separates concerns across frontend, backend, and database layers, thereby enhancing maintainability, scalability, and performance.

The frontend of the application is developed using React, a widely adopted JavaScript library for building dynamic and responsive user interfaces. React's component-based architecture facilitates the creation of reusable UI elements and efficient state management, allowing users to interact with the system in real time. Features such as event browsing, registration workflows, dashboards, and notifications are implemented with an emphasis on usability and responsiveness across devices.

On the backend, the system employs a hybrid architecture utilizing both FastAPI. FastAPI, a high-performance Python framework, is responsible for handling RESTful API endpoints related to event management, user registration, and data retrieval. Its asynchronous capabilities and automatic validation mechanisms ensure efficient request handling and reduced latency.

The data layer is managed using MySQL, a relational database management system known for its reliability and structured data handling capabilities. The database schema is designed to maintain normalized tables for entities such as users, events, subjects, topics, schedules, and progress tracking. Relationships between these entities are enforced through foreign keys and constraints to ensure data integrity and consistency. Additionally, indexing and query optimization techniques are applied to improve retrieval performance, especially in scenarios involving large datasets and concurrent access.

Security and access control are integral components of the system design. Authentication and authorization mechanisms are implemented to ensure that only authorized users can perform specific actions, such as creating events or modifying schedules. Data validation, secure API endpoints, and role-based access control (RBAC) contribute to maintaining system integrity and preventing unauthorized access.

Furthermore, the platform is designed with scalability in mind, allowing it to handle increasing numbers of users and events without significant degradation in performance. The use of RESTful APIs facilitates integration with other systems and services, enabling future extensibility such as mobile applications or third-party integrations. Logging and error-handling mechanisms are also incorporated to ensure system reliability and ease of maintenance.

By consolidating disparate processes into a unified platform, the proposed system significantly enhances coordination, reduces redundancy, and improves the overall efficiency of campus operations. It provides a streamlined workflow for event management while ensuring that users have access to accurate and real-time information. Ultimately, this project aims to transform the way campus activities are managed by leveraging modern web technologies and best practices in software engineering.

Chapter 2

Existing System Study

1. Manual Notice Board System

- Events and announcements are displayed on physical notice boards
- No real-time updates or digital access
- Students may miss important information

2. Email-Based Communication

- Event details shared through institutional emails
- Messages can be overlooked or ignored
- No centralized tracking of events or participation

3. Messaging Platforms (WhatsApp/Telegram Groups)

- Used for quick communication of events and updates
- Information gets lost in chats due to high message volume
- No structured event management or record keeping

4. Google Forms & Sheets-Based Management

- Used for event registrations and data collection
- Requires manual handling of responses and updates

- Lacks automation and integration with event scheduling

5. Basic College Websites

- Static pages used to display event information
- Limited interactivity and no real-time updates
- No user-specific features like registration or dashboards

Key Limitations of Existing Systems

- Lack of centralization
- Poor communication and coordination
- No role-based access control
- High manual workload
- Increased chances of errors and missed updates

Chapter 3

Problem Statement, Objective & Scope

Problem Statement:-

To design and implement a centralized, secure, and user-friendly campus event management system for efficient coordination, resource booking, and communication, using modern web technologies such as React, and database management systems.

Description:

The current system for managing campus events and resources is scattered across multiple platforms, leading to poor coordination, duplicate bookings, and missed announcements. This creates inefficiencies for both organizers and participants.

To overcome these issues, the proposed solution is a centralized event management system that integrates event creation, resource booking, and communication into a single platform. It ensures better coordination, improved accessibility, and efficient management using modern web technologies.

Objective: -

- To develop a role-based Campus Resource and Event Portal for admins, faculty, and students.
- To streamline event creation, booking, and notice management through automation.
- To ensure secure access and data integrity using Flask and MySQL.
- To provide intuitive dashboards that enhance user experience and communication.
- To centralize all campus events and resources, reducing manual work and scheduling conflicts.

Scope: -

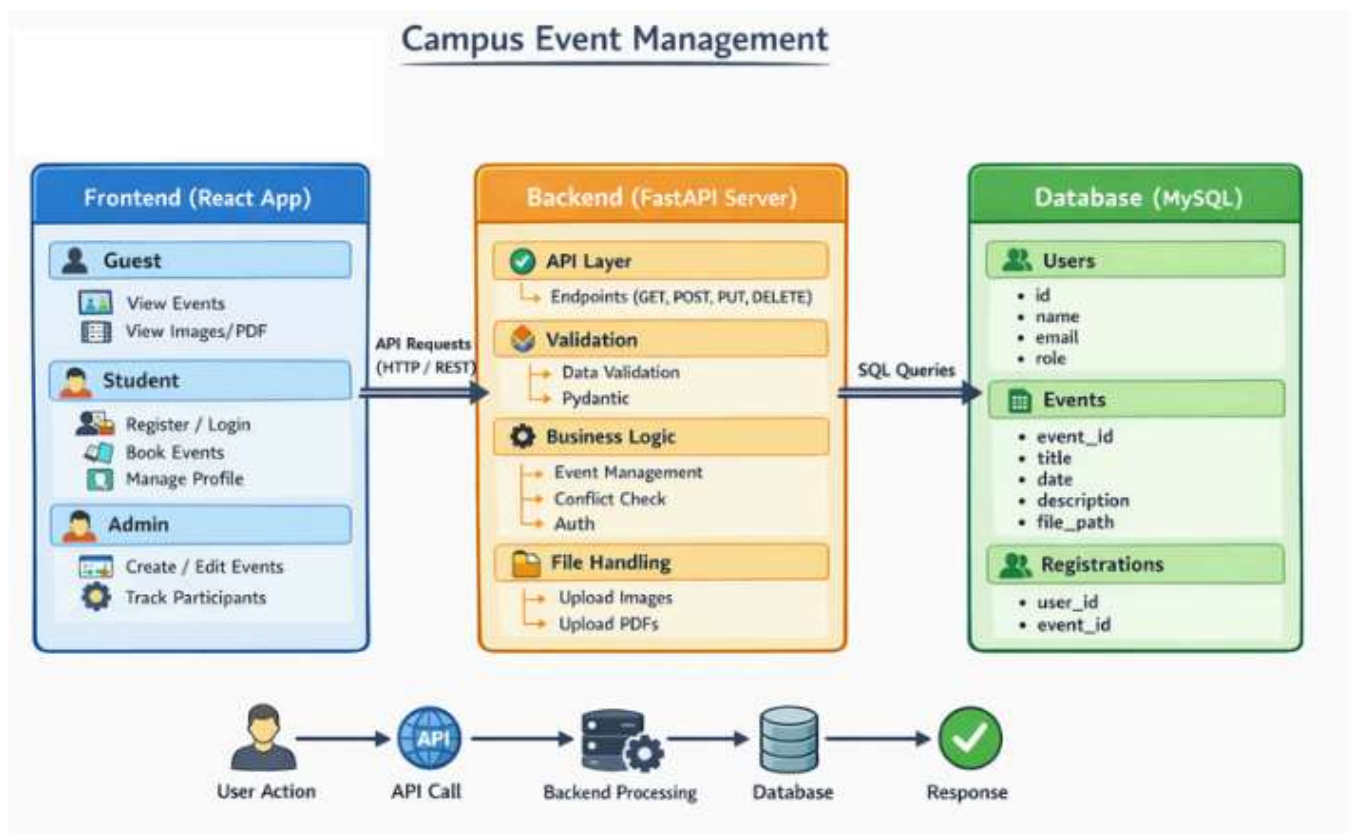
- Covers management of **academic and co-curricular** events within the campus
- Supports **multiple user roles (Admin, Student)** with defined permissions and access levels
- Enables **administrators** to **create, approve, monitor, and control** campus events and notices
- Provides **students** with facilities to **view, search, register, and participate** in campus activities
- Enhances campus connectivity and engagement by centralizing all information in one platform
- Maintains **data consistency** using a **relational database** for users, events, and bookings
- Prevents **event conflicts and duplicate bookings** through validation logic
- Ensures **secure access** through **authentication and authorization** mechanisms
- Acts as a **centralized hub** to **improve campus engagement** and **reduce manual workload**

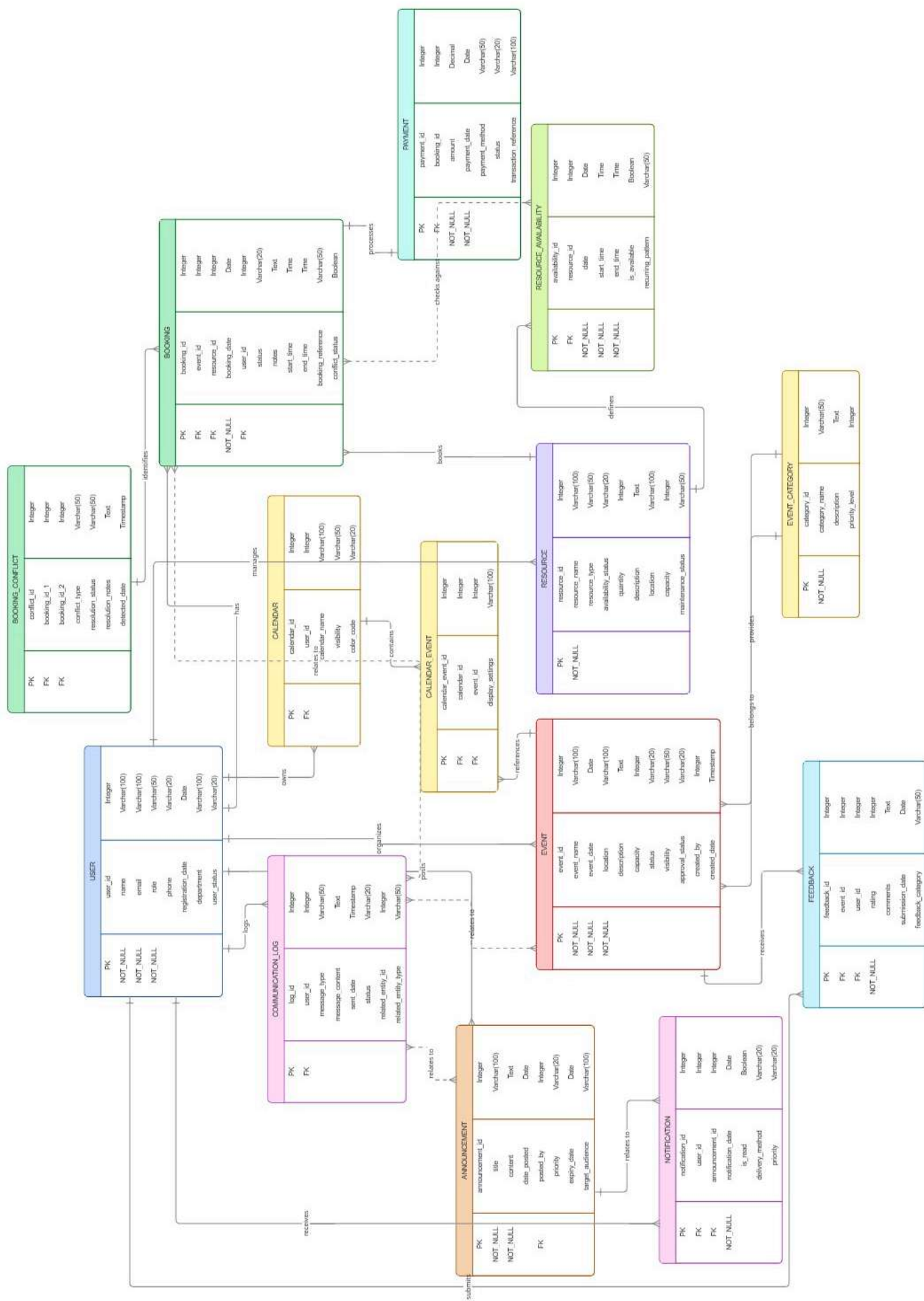
Chapter 4

Proposed System Architecture

Architecture diagram, ER diagram compulsory with explanation (each diagram on new page)

- o Analysis/Framework/ Algorithm
- o Design details
- o Methodology (your approach to solve the problem) Proposed System





1. Core Entity:User

The USER table is central to the system.

Key attributes:

- user_id (PK)
- name, email, role, phone
- registration_date, department, status

What users do:

- Create and manage calendars
- Book resources
- Post announcements
- Send communications
- Receive notifications and give feedback

2. Calendar & Events System

CALENDAR

- Each user owns one or more calendars
- Stores visibility and display preferences

EVENT

Represents actual events.

Attributes:

- event_name, date, location

- capacity, status, visibility
- created_by (user)

CALENDAR_EVENT (junction table)

- Links events to calendars (many-to-many)
- Allows an event to appear in multiple calendars

EVENT_CATEGORY

- Categorizes events (e.g., meeting, workshop)
- Each event belongs to a category

3. Resource Management

RESOURCE

Represents assets like rooms, equipment, etc.

Attributes:

- resource_name, type
- location, capacity
- availability_status, maintenance_status

RESOURCE_AVAILABILITY

- Defines when a resource is available
- Includes date, start/end time, recurring patterns

4. Booking System

BOOKING

Central entity for reserving resources.

Attributes:

- booking_id, event_id, resource_id, user_id
- booking_date, start_time, end_time
- status, notes, reference
- conflict_status (boolean)

BOOKING_CONFLICT

- Detects and stores conflicts between bookings
- Links two conflicting bookings
- Tracks resolution status and notes

PAYMENT

- Associated with bookings
- Stores amount, payment method, status, transaction reference

5. Communication & Notifications**COMMUNICATION_LOG**

Tracks messages sent by users.

Attributes:

- message_type, content
- sent_date, status
- related_entity_id + type (polymorphic link to events, bookings, etc.)

ANNOUNCEMENT

- Official messages posted by users
- Includes title, content, target audience, expiry

NOTIFICATION

- Sent to users (often triggered by announcements)
- Tracks delivery method, priority, read status

6. Feedback System

FEEDBACK

- Users provide feedback on events

Attributes:

- rating, comments
- submission_date, category

Key Relationships (Simplified)

User Relationships

- USER → CALENDAR (owns)
- USER → BOOKING (creates)
- USER → ANNOUNCEMENT (posts)
- USER → COMMUNICATION_LOG (logs messages)
- USER → FEEDBACK (gives feedback)

Event Relationships

- EVENT ↔ CALENDAR (via CALENDAR_EVENT)
- EVENT → EVENT_CATEGORY
- EVENT → FEEDBACK

- EVENT $\rightarrow$ BOOKING

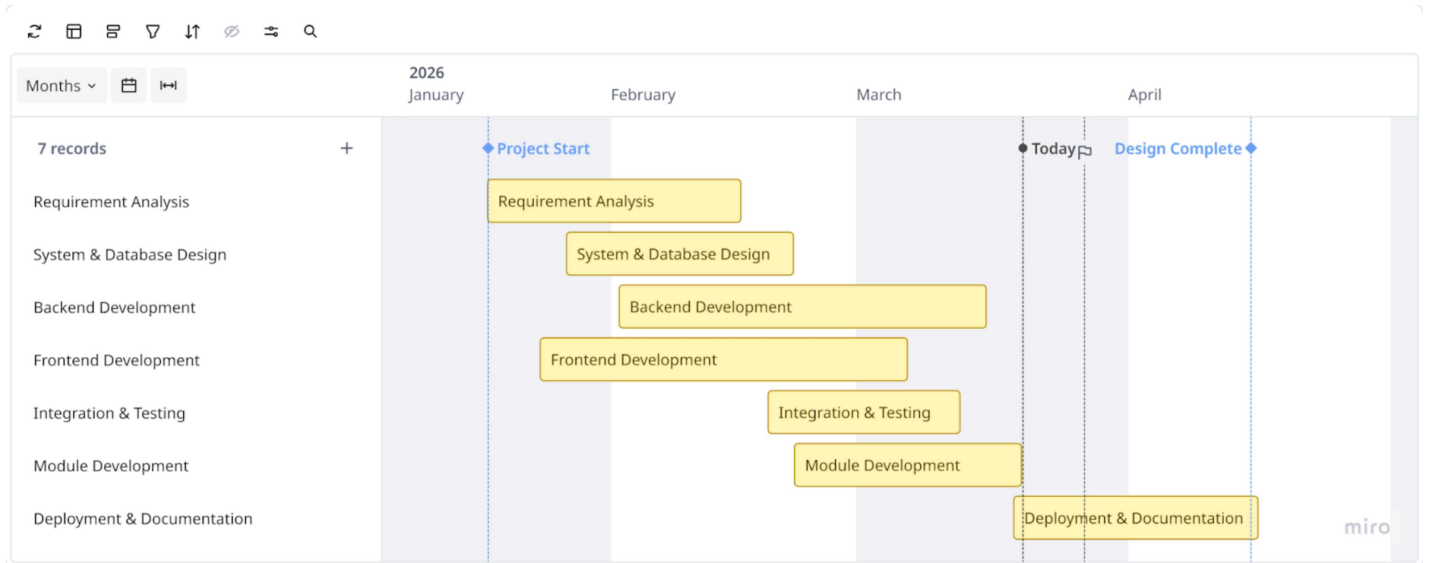
Resource & Booking Relationships

- RESOURCE $\rightarrow$ BOOKING
- RESOURCE $\rightarrow$ RESOURCE_AVAILABILITY
- BOOKING $\rightarrow$ PAYMENT
- BOOKING $\leftrightarrow$ BOOKING_CONFLICT

Chapter 5

Project Planning

Gantt chart



1. Requirement Analysis (Week 1–2)

Understand system needs: users, events, bookings, resources, and communication features.

2. System & Database Design (Week 3–4)

Create ER diagram (like yours), define schema, relationships, and system architecture.

3. Backend Development (Week 5–10)

Develop APIs and logic for:

- Users
- Events & calendars
- Resources
- Bookings & conflicts
- Payments

4. Frontend Development (Week 6–11)

Build UI for dashboards, booking forms, calendars, notifications, etc.

5. Module Development (Week 6–12)

Parallel development of key modules:

- User management
- Event handling
- Resource tracking
- Booking system
- Notifications & feedback

6. Integration & Testing (Week 11–14)

Combine frontend + backend, test APIs, fix bugs, validate workflows.

7. Deployment & Documentation (Week 15–16)

Deploy system and prepare user/admin documentation.

Experimental Setup

6.1 Technology Stack Used

The *Campus Resource & Event Portal* is developed using a modern full-stack architecture to ensure efficiency, scalability, and performance.

Frontend:

The frontend of the system is developed using **React.js**, which enables the creation of a dynamic and responsive user interface. React's component-based architecture allows efficient state management and reusable UI components. It is used to design features such as event dashboards, registration forms, and user interfaces.

Backend:

The backend is implemented using **FastAPI**, a high-performance Python framework used for building RESTful APIs. FastAPI handles request processing, business logic, authentication, and communication between the frontend and database. It ensures fast execution and efficient handling of multiple requests.

Database:

The system uses **MySQL**, a relational database management system, to store structured data such as user details, event information, bookings, and registrations. MySQL ensures data consistency, integrity, and efficient querying.

Tools & Technologies:

- **SQLAlchemy** – Used as ORM for database interaction
- **Postman** – API testing and debugging
- **Git & GitHub** – Version control
- **VS Code** – Development environment

6.2 System Environment

Hardware Requirements:

- Processor: Intel i3 or above
- RAM: Minimum 4GB (8GB recommended)
- Storage: At least 500MB free space

Software Requirements:

- Operating System: Windows/Linux/macOS
- Web Browser: Google Chrome or Microsoft Edge
- Code Editor: Visual Studio Code
- Backend Server: Localhost (FastAPI server)

6.3 Database Design

The database is designed using a **relational model** to maintain structured and consistent data.

Key Tables:

- **User Table** – Stores user details such as ID, name, email, and role
- **Event Table** – Contains event information like title, date, venue, and category
- **Category Table** – Classifies different types of events
- **Participant Table** – Stores event registration details
- **Resource Table** – Maintains information about available resources
- **Booking Table** – Handles resource booking details

ER Diagram (Explanation):

- A **User** can create multiple **Events**
- An **Event** belongs to a **Category**
- A **User** can register for multiple **Events**
- A **Booking** connects **User**, **Event**, and **Resource**

Normalization:

The database follows **Third Normal Form (3NF)**:

- Eliminates redundant data
- Ensures all attributes depend on primary keys
- Improves data consistency and reduces anomalies

6.4 Module Description

The system is divided into different modules for better functionality and management.

1. User Management Module

- Handles user registration and login
- Provides role-based access (Admin, Faculty, Student)
- Ensures secure authentication

2. Event Management Module

- Allows creation, updating, and deletion of events
- Displays event details to users
- Categorizes events

3. Resource Management Module

- Manages campus resources such as rooms and equipment
- Tracks availability and usage

4. Booking Module

- Allows users to book resources
- Prevents double booking using validation
- Maintains booking records

5. Notification Module

- Provides updates about events and bookings
- Displays announcements

6. Feedback Module

- Collects feedback from users
- Stores ratings and suggestions

6.5 Working Procedure

1. User logs into the system
2. Admin or faculty creates an event
3. System checks resource availability
4. Users view and register for events
5. Booking is validated and stored in the database
6. Notifications are sent to users
7. Feedback is collected after event completion

6.6 Testing

The system is tested using different testing techniques:

- **Unit Testing** – Individual modules are tested
- **Integration Testing** – Ensures proper communication between frontend and backend

- **API Testing** – Tested using Postman
- **User Testing** – Validates usability and functionality

Test Cases Include:

- User login and registration
- Event creation and display
- Booking validation
- Conflict detection

6.7 Performance Considerations

- Fast API response using FastAPI
- Efficient database queries using indexing
- Reduced redundancy through normalization
- Scalable architecture to handle multiple users

Chapter 7

Implementation Details

1. Frontend–Backend Integration

The project follows a full-stack architecture where the frontend and backend communicate through RESTful APIs. The frontend, developed using React and Vite, interacts with the backend by sending HTTP requests using the Fetch API. These requests are directed to FastAPI endpoints hosted at `http://localhost:8000/api/...`

User actions such as login, registration, event browsing, and participation trigger API calls from the frontend. The backend processes these requests, performs necessary operations, and returns responses in JSON format. The frontend then dynamically updates the user interface based on the received data, ensuring a seamless and real-time user experience.

2. Database Integration

The backend is integrated with a MySQL relational database using SQLAlchemy ORM. The database stores structured data related to users, events, categories, and participants.

- User Table: Stores user credentials and role information
- Event Table: Stores event details such as title, date, venue, and category
- Category Table: Manages classification of events
- Participant Table: Maintains records of user registrations for events

SQLAlchemy manages the connection, query execution, and data mapping between Python objects and database tables. This ensures efficient data handling, consistency, and scalability.

3. SQL Queries & Implementation

The system performs various database operations using SQL queries generated through SQLAlchemy ORM. These include:

- Insertion Queries:
Used for adding new users, events, and participant registrations

(e.g., registering a new user or creating an event)

- **Selection Queries:**
Retrieve data such as event lists, user details, and participant records
(e.g., fetching all events or events by category)
- **Update Queries:**
Modify existing records like updating user profiles or event details
- **Deletion Queries:**
Remove events or user records when required
- **Validation Queries:**
Ensure no duplicate bookings or event conflicts by checking date and time conditions before insertion

4. Implementation Workflow

1. The user interacts with the frontend (e.g., registers for an event).
2. The frontend sends a request to the backend API.
3. FastAPI processes the request and interacts with the database using SQLAlchemy.
4. The database executes the query and returns the result.
5. The backend sends a response to the frontend.
6. The frontend updates the UI accordingly.

5. Key Features of Integration

- Real-time data fetching and updates
- Secure data handling through API-based communication
- Efficient database operations using ORM
- Prevention of event conflicts through validation logic
- Smooth user experience with dynamic UI updates

Chapter 8

Result

SDG 4 – Quality Education

The Campus Resource & Event Portal contributes to quality education by improving access to academic events, workshops, seminars, and learning resources. It ensures that students are well-informed about opportunities that enhance their knowledge and skills, thereby supporting continuous learning and academic growth.

SDG 9 – Industry, Innovation & Infrastructure

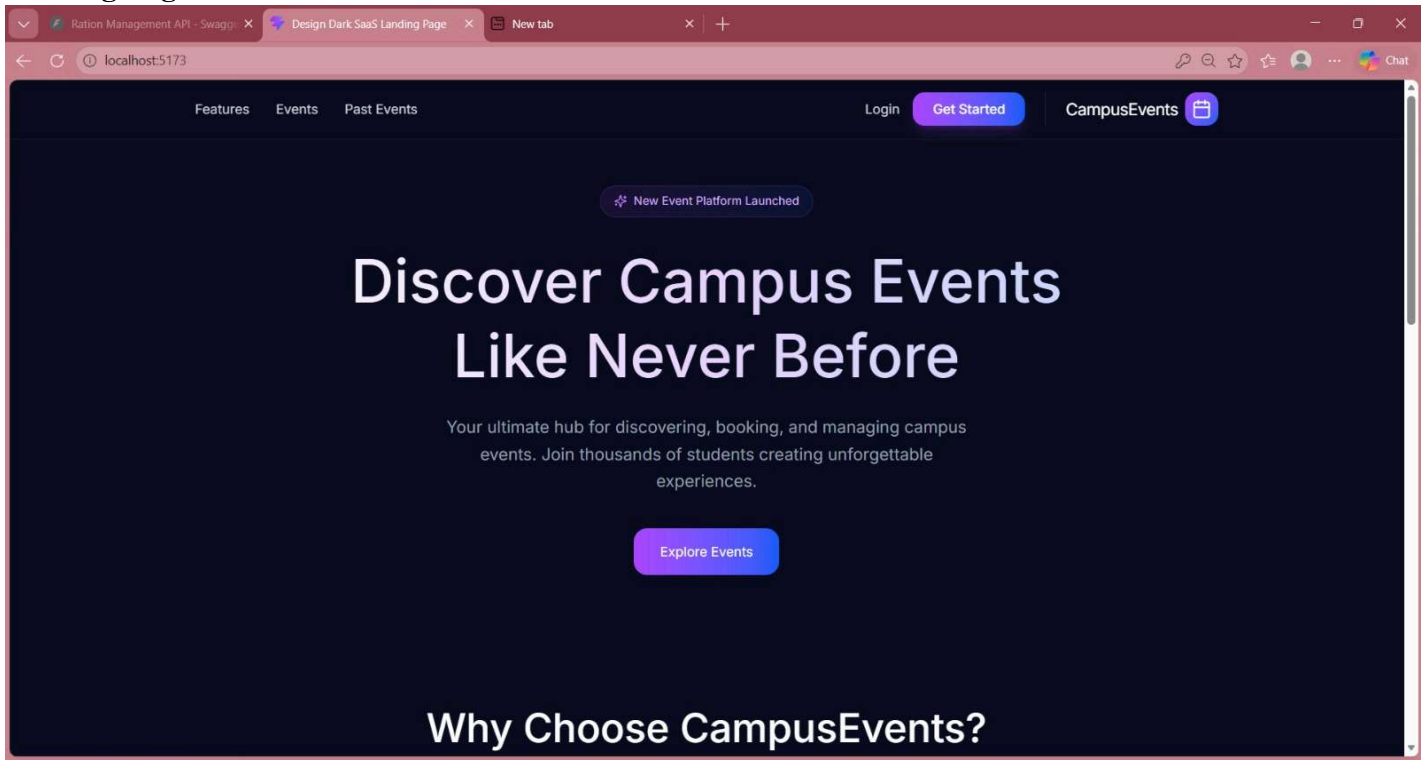
The project promotes innovation by integrating modern digital technologies into campus management. By using a web-based platform built with advanced technologies, it helps modernize institutional infrastructure, making processes more efficient, scalable, and technology-driven.

SDG 16 – Peace, Justice & Strong Institutions

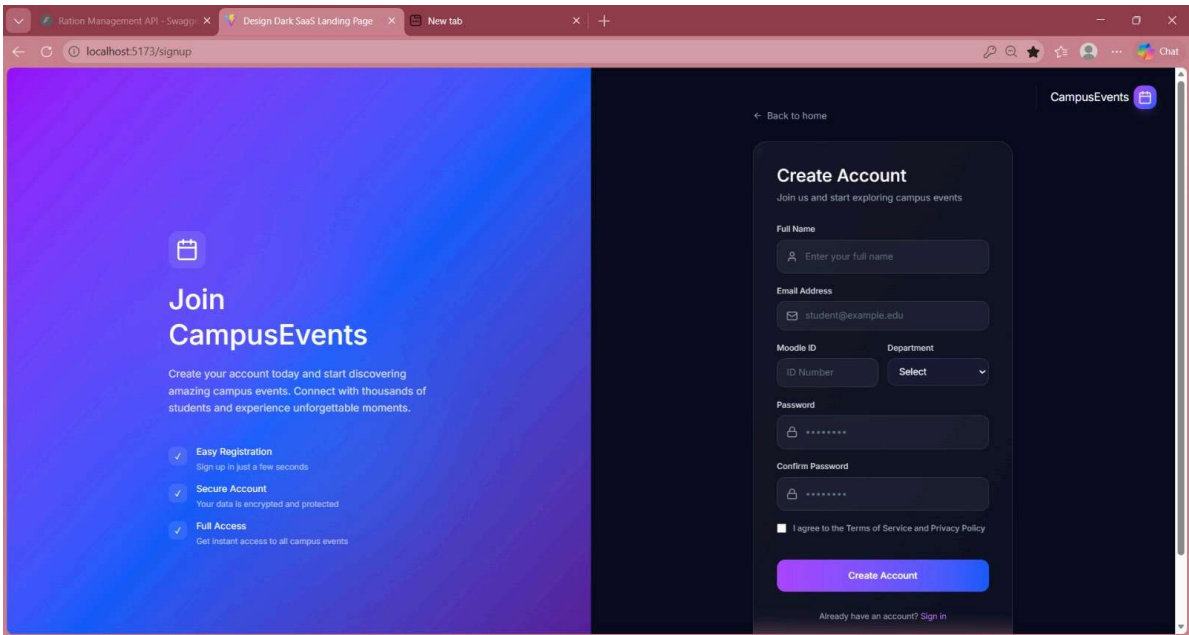
The portal enhances transparency and accountability in institutional management by providing a structured and centralized system for handling events and resources. Role-based access and secure operations ensure fairness, reliability, and trust within the campus ecosystem.

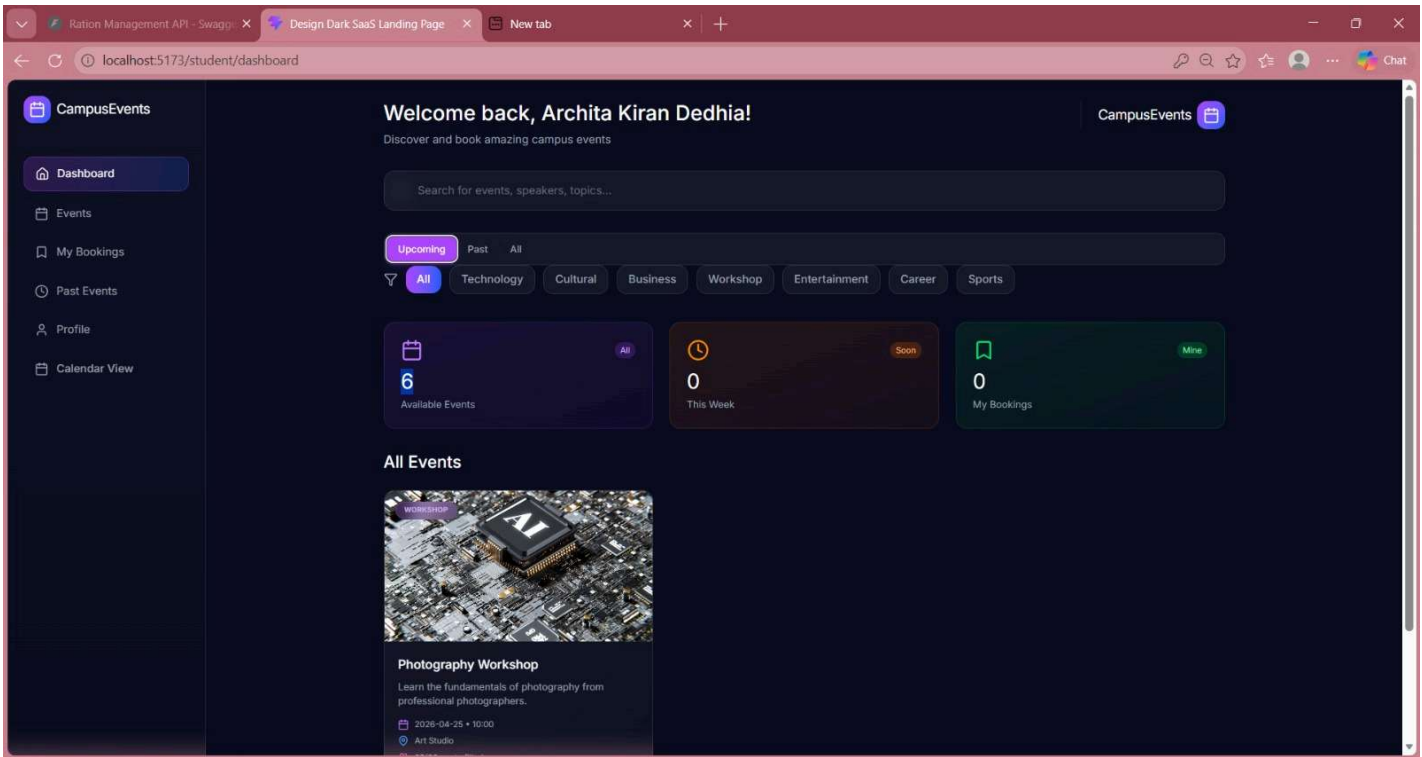
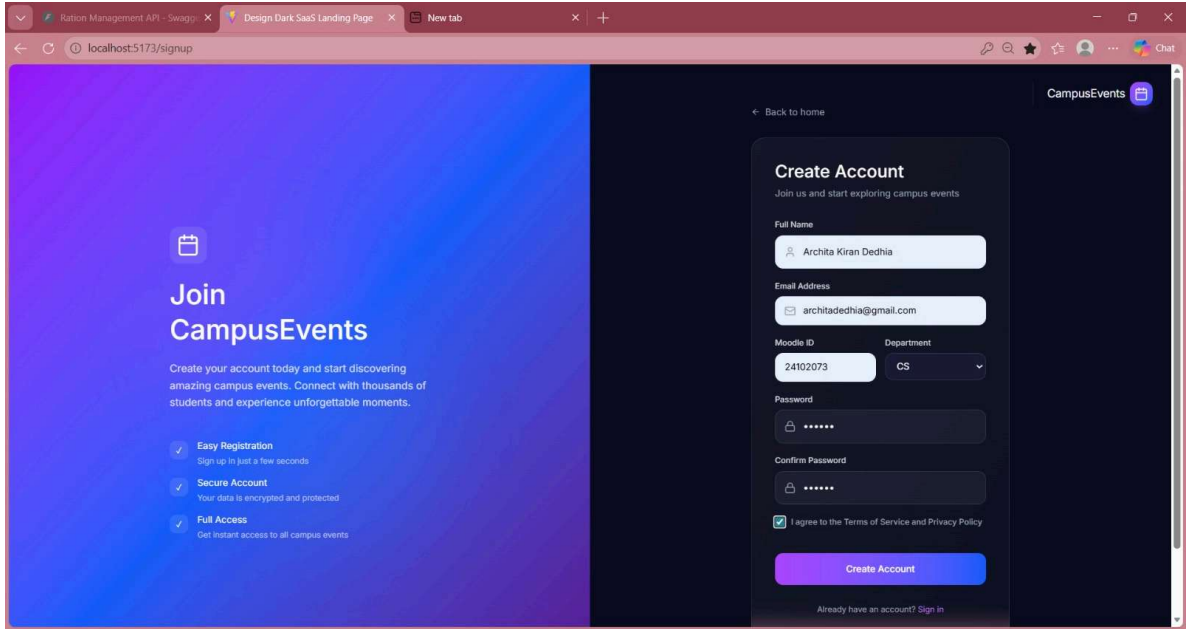
Frontend:

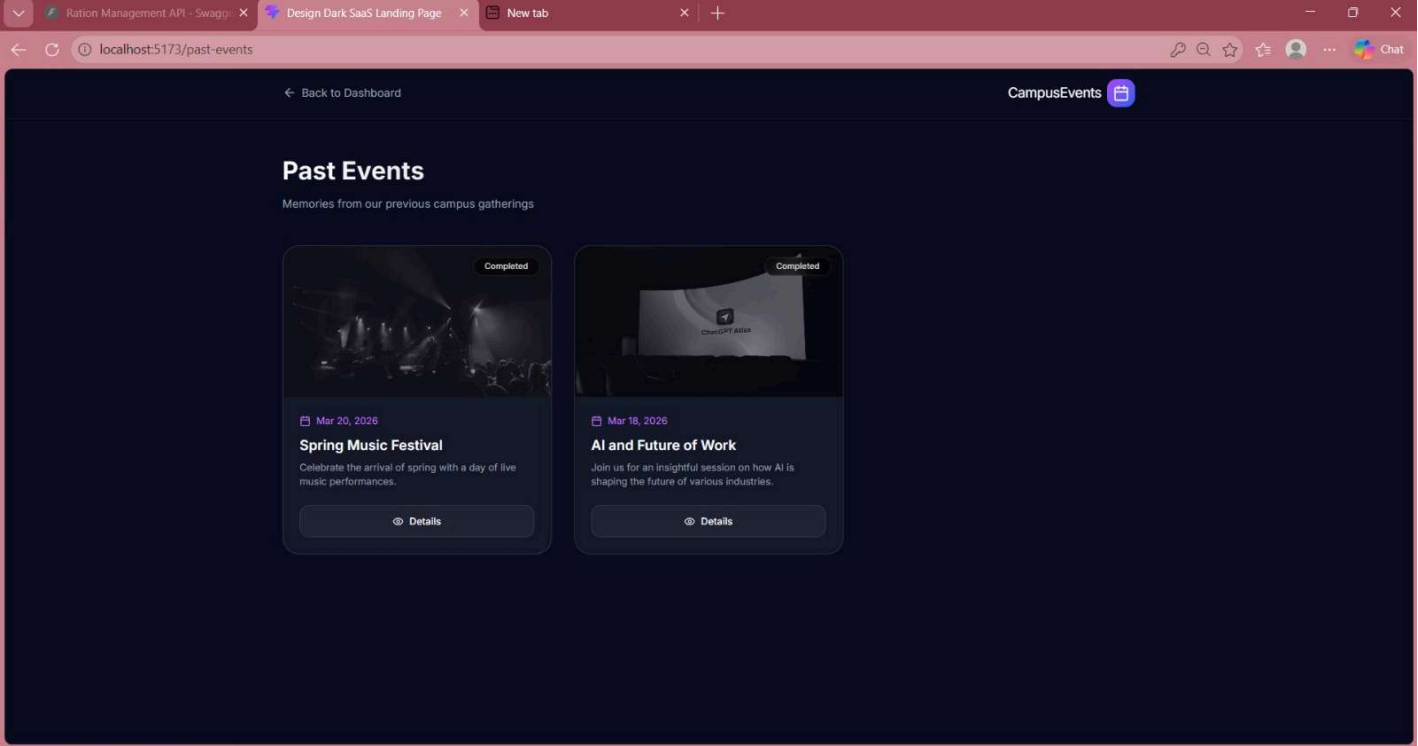
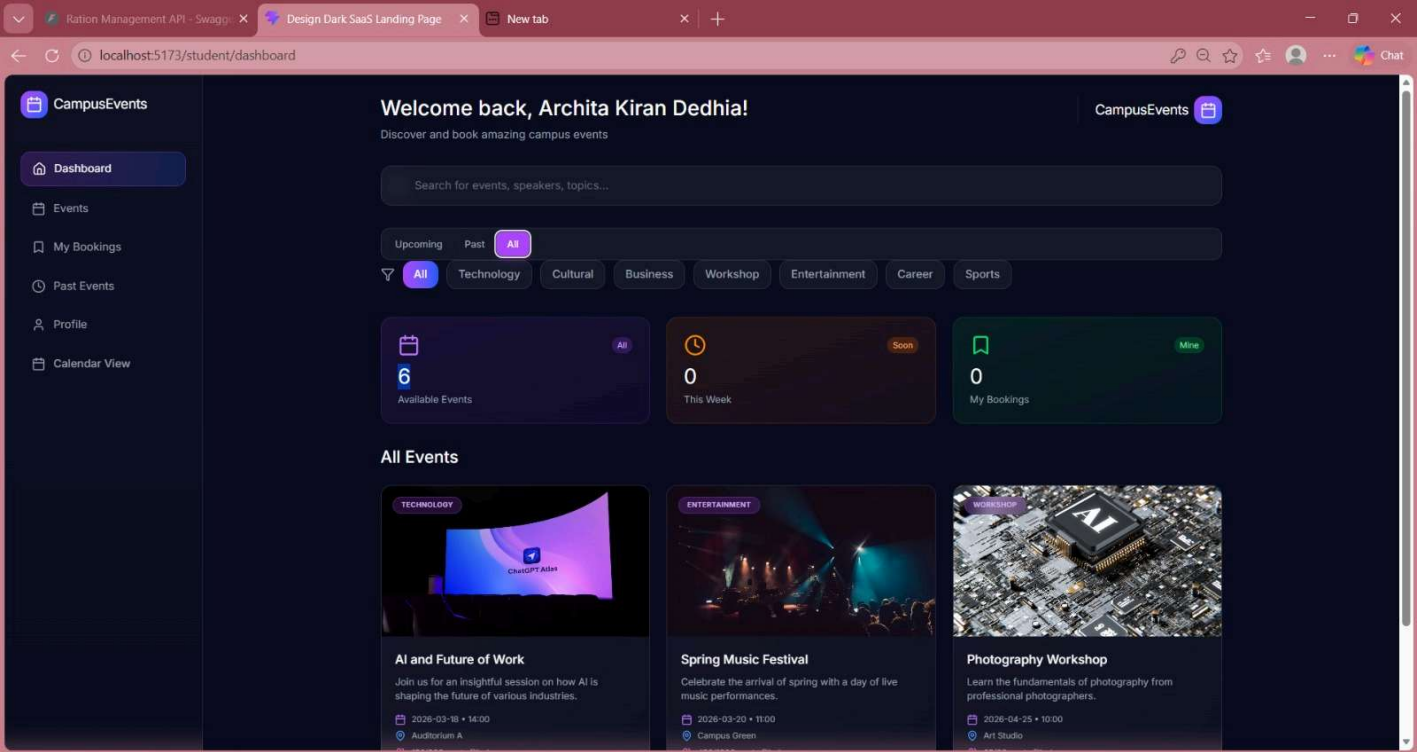
Landing Page:

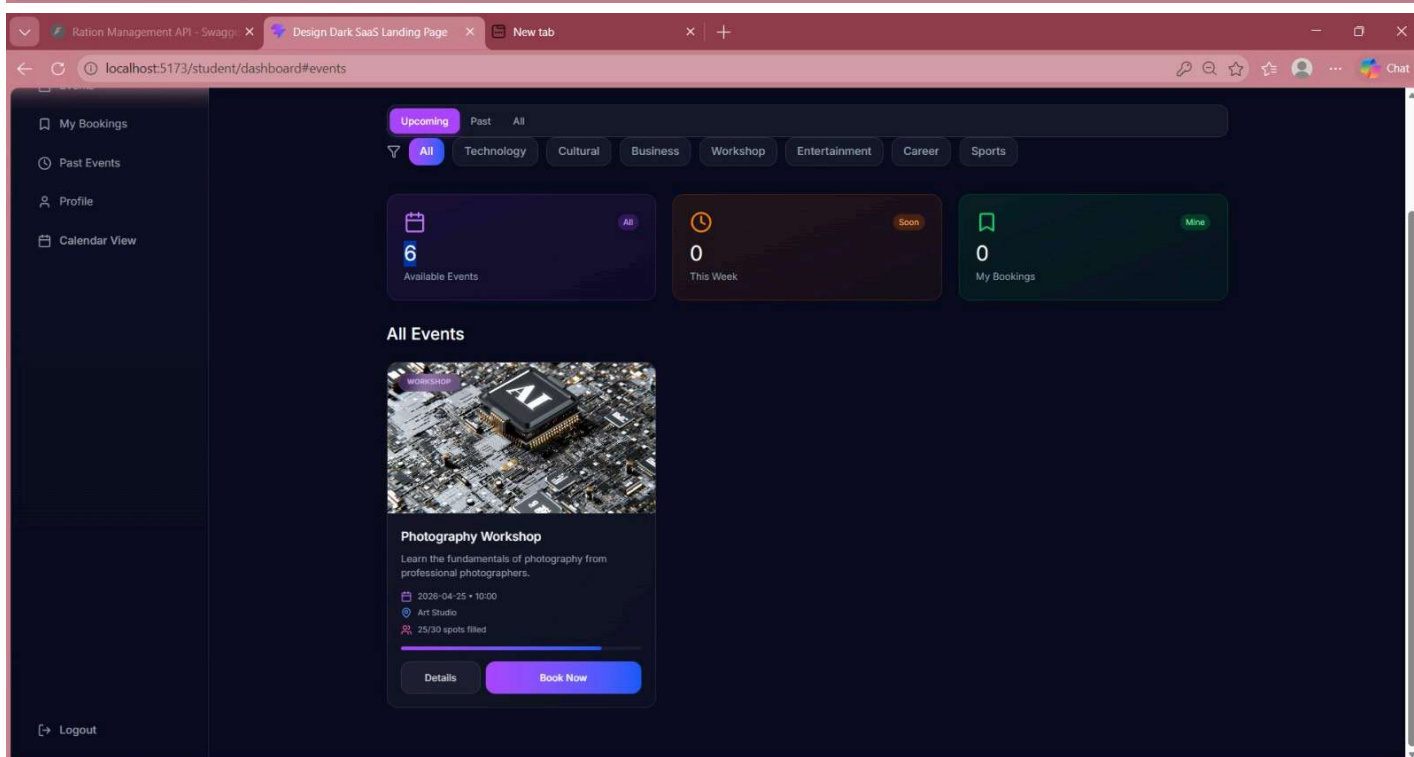
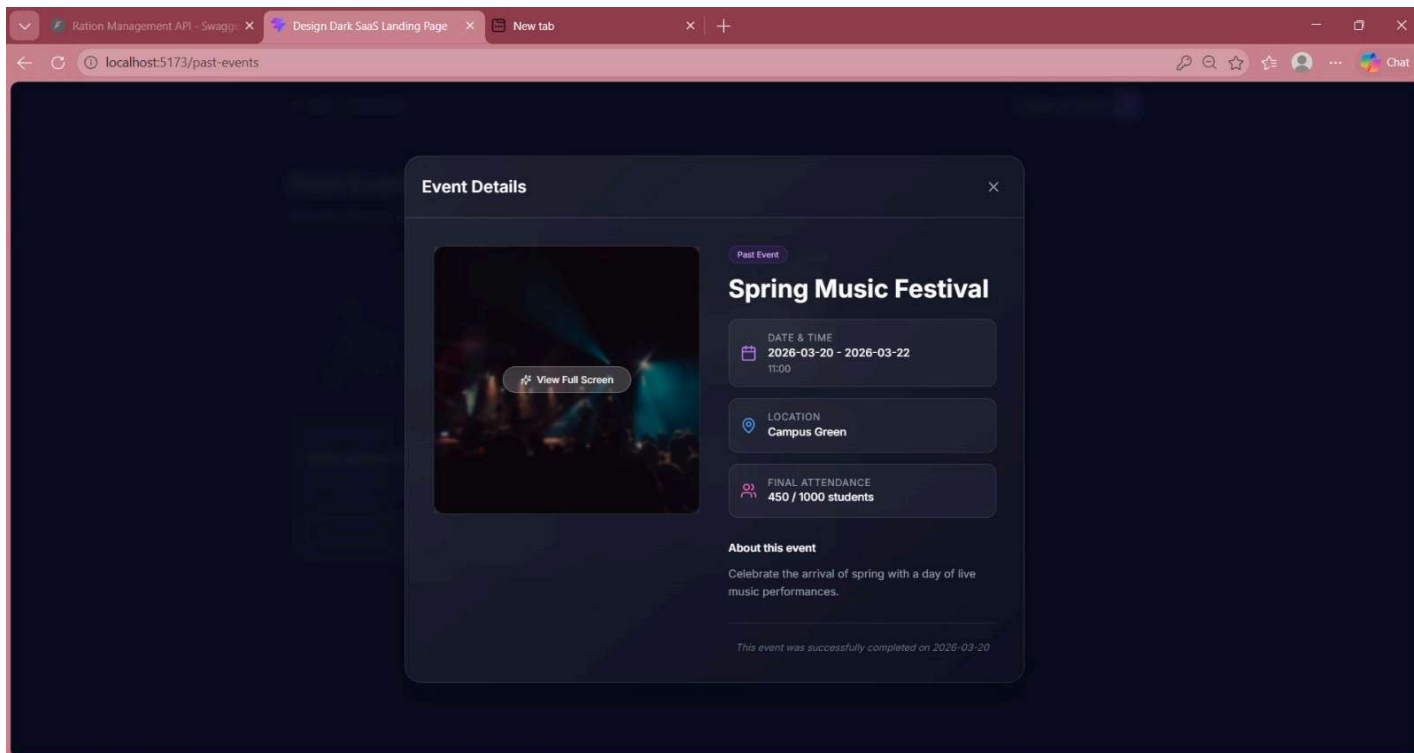


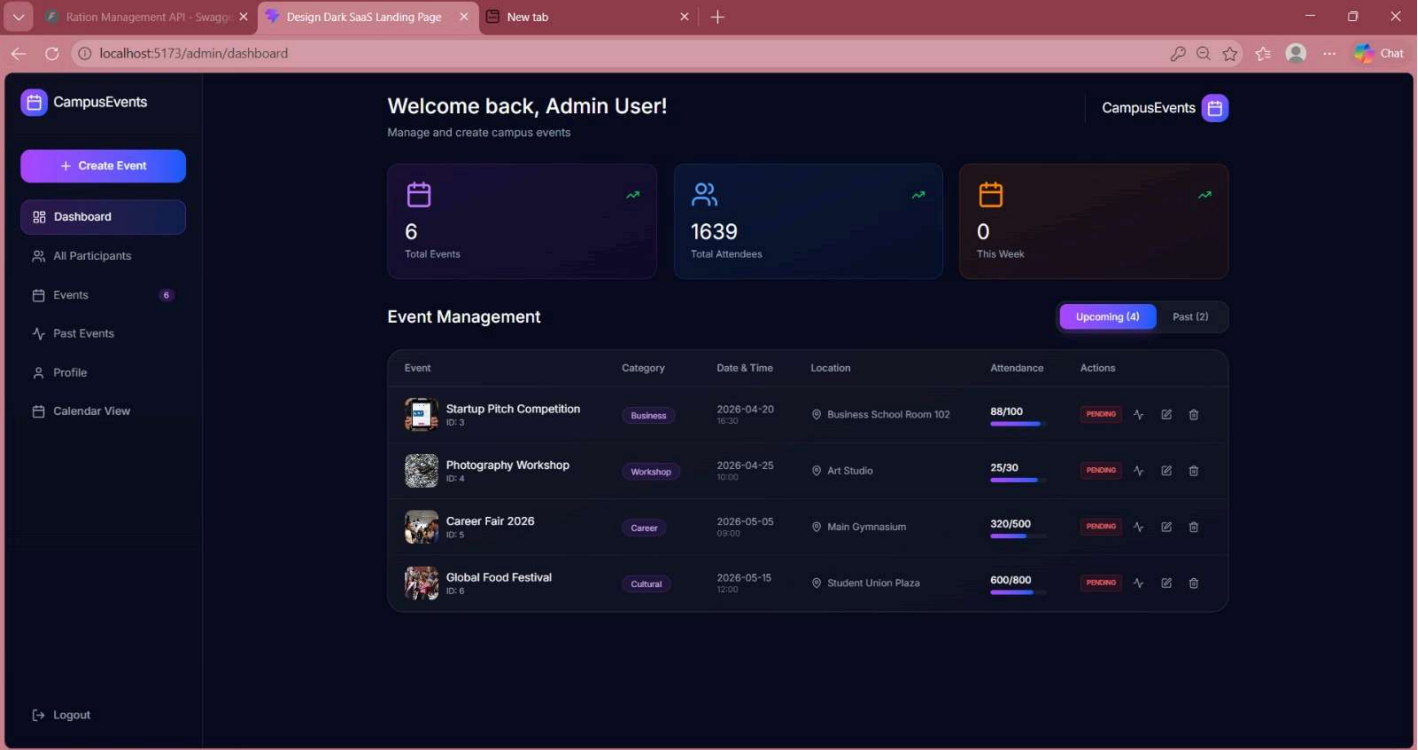
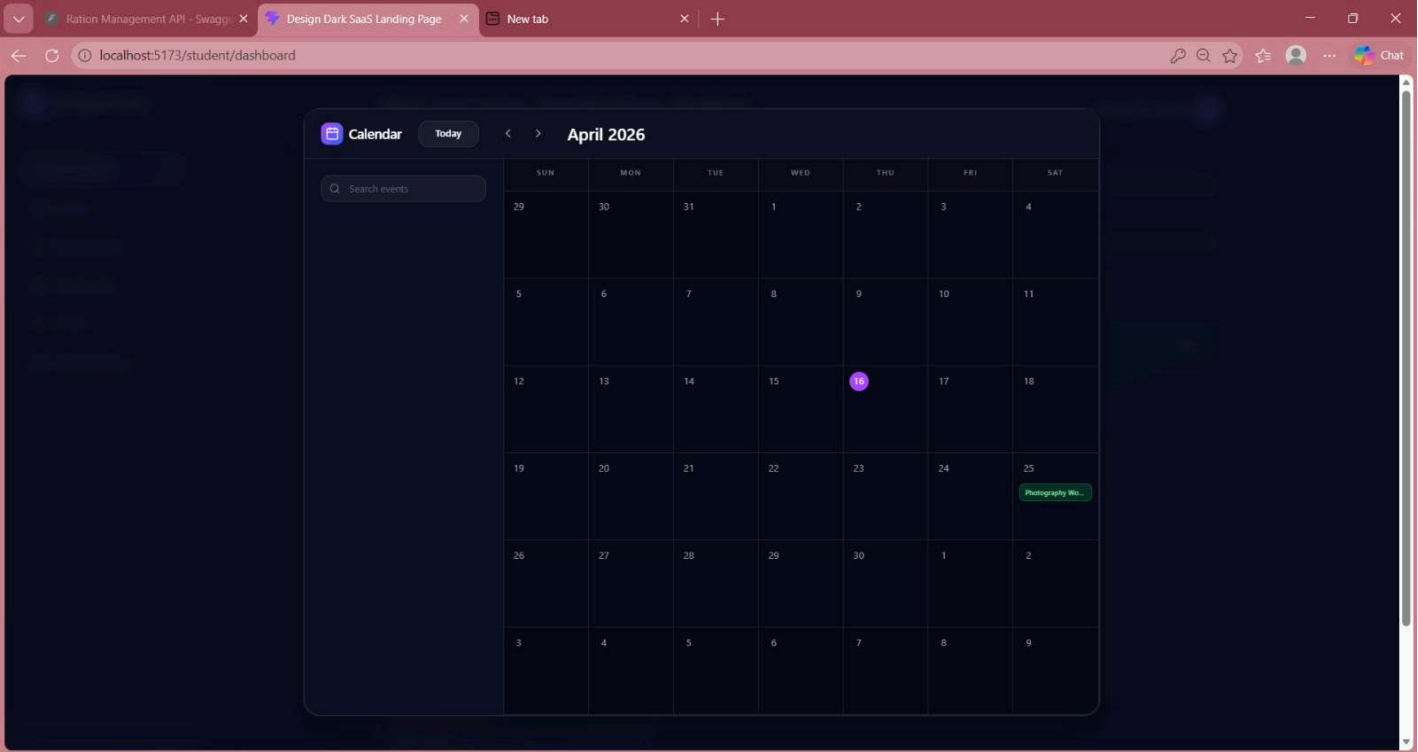
Login Page:

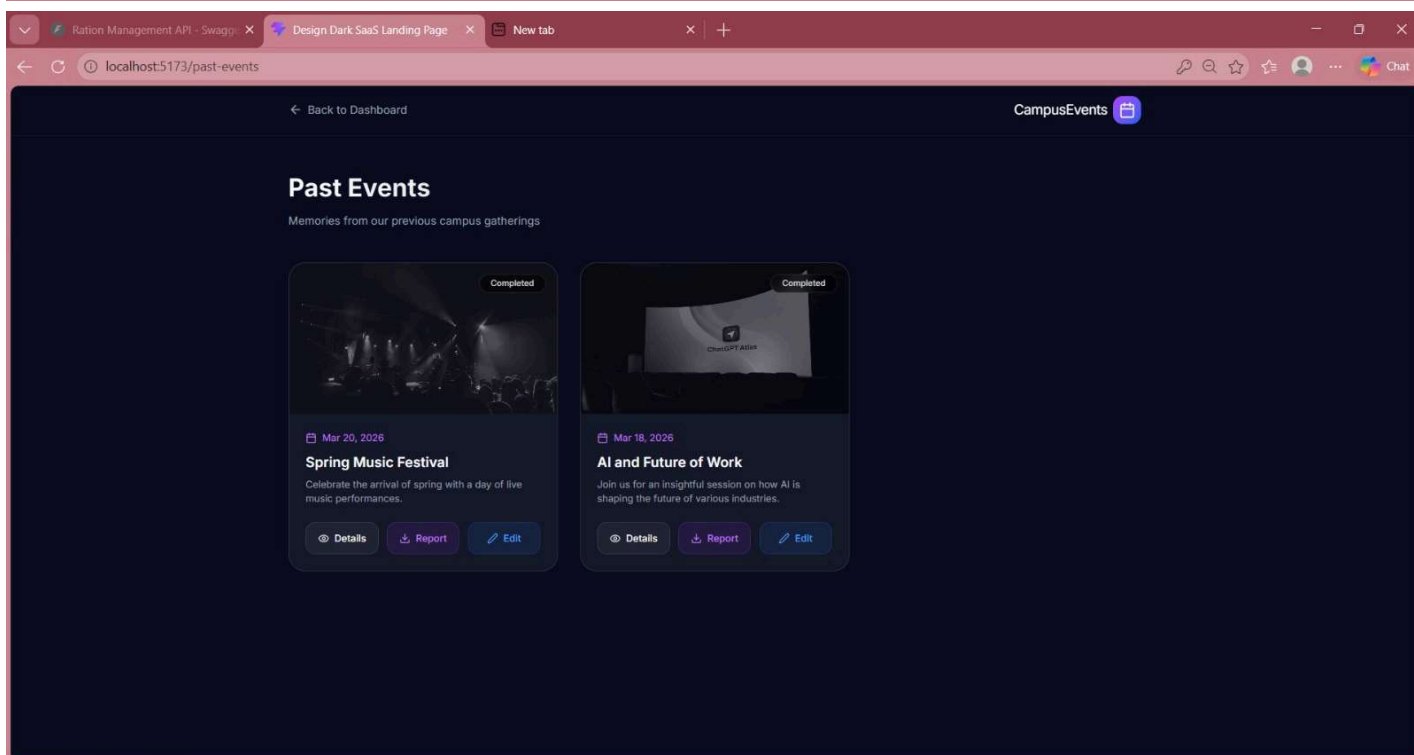
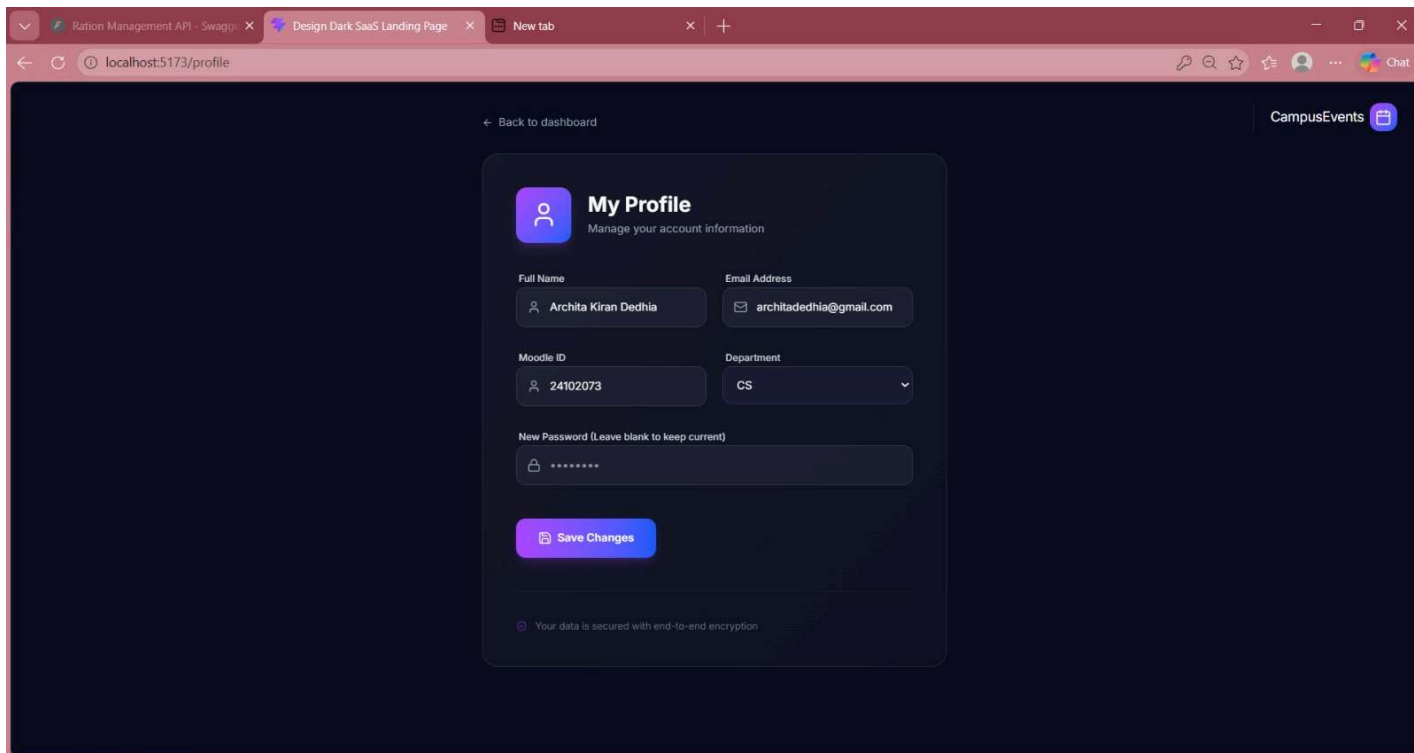




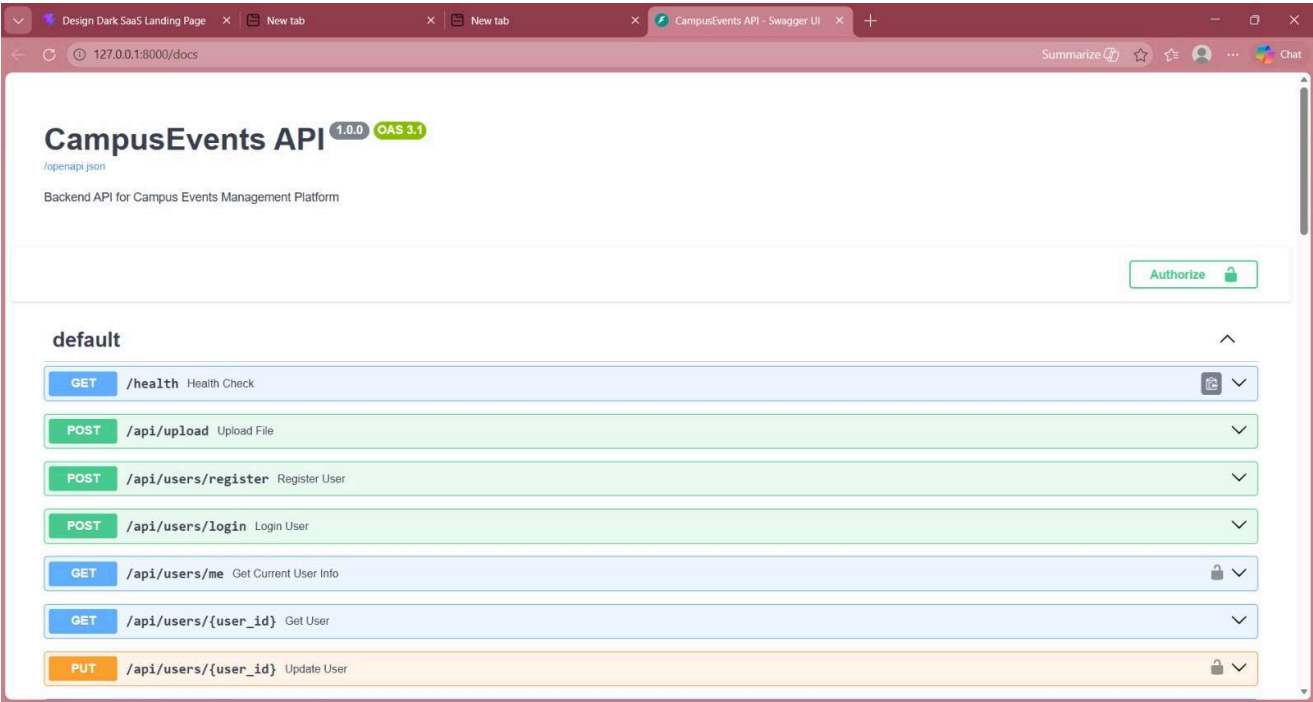
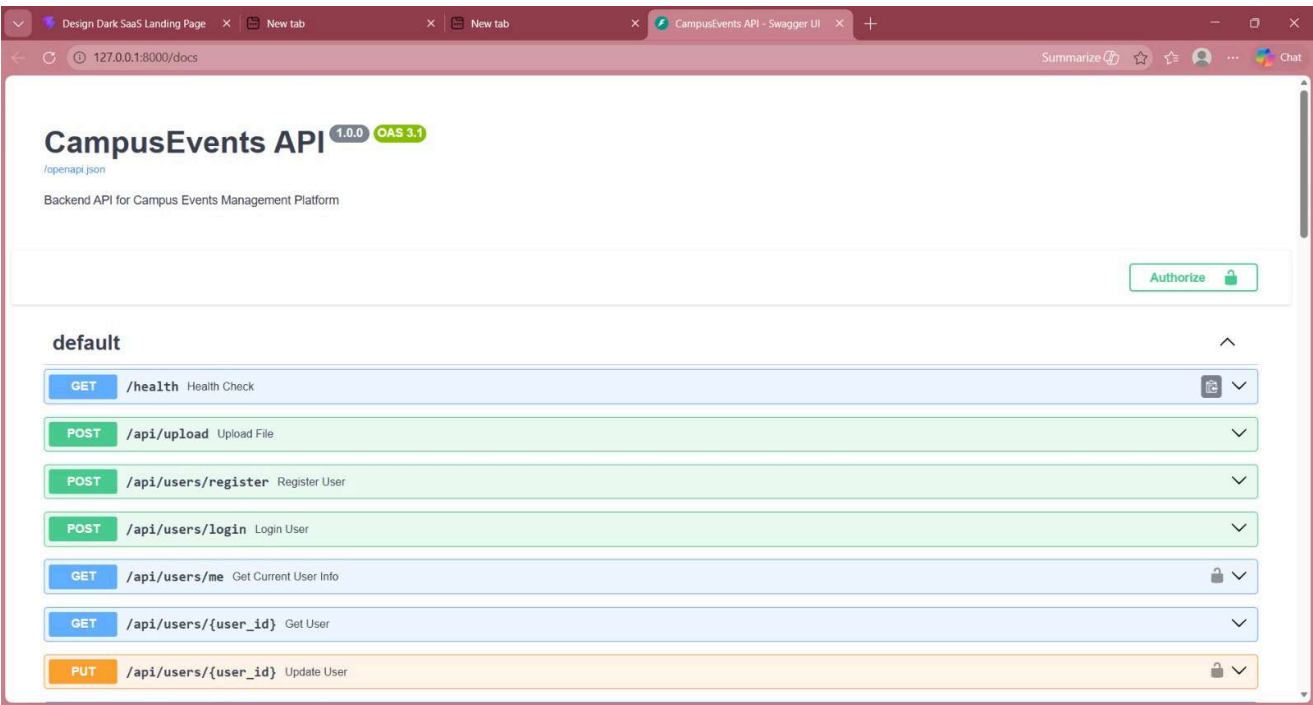








Backend:



Design Dark SaaS Landing Page | New tab | New tab | CampusEvents API - Swagger UI | +

127.0.0.1:8000/docs

Summarize | ☆ | ⚙️ | 👤 | ... | Chat

Authorize

GET	/api/participants/user/{user_id}	Get User Events	✓
GET	/api/participants/event/{event_id}	Get Event Participants	✓
DELETE	/api/participants/{participant_id}	Cancel Booking	🔒 ✓
GET	/api/admin/participants	Get All Organizer Participants	🔒 ✓
GET	/api/admin/analytics	Get Admin Analytics	🔒 ✓

Schemas

- AdminAnalytics > Expand all object
- Body_upload_file_api_upload_post > Expand all object
- CategoryCreate > Expand all object
- CategoryOut > Expand all object
- EventCreate > Expand all object

Design Dark SaaS Landing Page | New tab | New tab | CampusEvents API - Swagger UI | +

127.0.0.1:8000/docs

Summarize | ☆ | ⚙️ | 👤 | ... | Chat

Authorize

- EventCreate > Expand all object
- EventDetail > Expand all object
- EventImageOut > Expand all object
- EventOut > Expand all object
- EventUpdate > Expand all object
- HTTPValidationError > Expand all object
- LoginResponse > Expand all object
- MessageResponse > Expand all object
- ParticipantCreate > Expand all object
- ParticipantDetail > Expand all object
- ParticipantOut > Expand all object

Design Dark SaaS Landing PageNew tabNew tabCampusEvents API - Swagger UI

127.0.0.1:8000/docs

Summarize

Authorize

MessageResponse > Expand all object

ParticipantCreate > Expand all object

ParticipantDetail > Expand all object

ParticipantOut > Expand all object

UserCreate > Expand all object

UserDetail > Expand all object

UserLogin > Expand all object

UserOut > Expand all object

UserUpdate > Expand all object

ValidationError > Expand all object

Chapter 9

Conclusion

The **Campus Resource & Event Portal** effectively addresses the challenges associated with traditional methods of managing campus events and resources. In many institutions, the use of disconnected platforms such as notice boards, emails, and messaging applications leads to poor communication, missed updates, and inefficient coordination. This project overcomes these issues by providing a centralized, web-based system that integrates all campus activities into a single platform.

The system offers role-based access for administrators, faculty, and students, ensuring that each user can perform tasks according to their responsibilities. Administrators can manage and approve events, faculty can organize and propose activities, and students can easily access information and participate in events. This structured approach enhances transparency, coordination, and overall efficiency within the campus environment.

The implementation of modern technologies such as React.js, FastAPI, and MySQL ensures that the system is responsive, scalable, and secure. Features like real-time updates, event search and filtering, and secure authentication improve usability and performance. Additionally, validation mechanisms help prevent scheduling conflicts and duplicate bookings, ensuring smooth event management.

Overall, the portal reduces manual workload by automating key processes and maintaining structured data. It improves communication, enhances user experience, and creates a more organized system for managing campus activities. Thus, the project provides a reliable and efficient solution that promotes better campus engagement and institutional productivity.

Chapter 10

References

- [1] Chaturvedi, A., Sharma, K., Dua, A., & Gupta, A. (2024). CU-Events: A Comprehensive Event Management System for University. International Journal for Research in Applied Science & Engineering Technology (IJRASET).
- [2] Rakesh, K., Sai Vishnu, K., & Nikhith Reddy, G. (2025). Unified Web Platform for Student-Campus Connectivity: A React.js-Based Approach. IJSDR (International Journal of Scientific Development and Research).
- [3] Authors as per IJRPR publication. (2024). EduLearn: A React Based EdTech Web Application. International Journal for Research Publication & Review (IJRPR).
- [4] Chaturvedi, A., Sharma, K., Dua, A., & Gupta, A. (2024). College Event Management System. International Journal of Engineering Science & Advanced Technology (IJESAT).